Jeopardy! Wagering Strategy White Paper

Jeopardy! Wagering Strategy: A Practical Model for Daily Double and Final Jeopardy Bets

Version 1.4 — 2026-07-23 (Version 1.4 is an infrastructure/tooling release — CI, a committed parity harness, and a docs index — with no change to the analysis since 1.3. The substantive statistical history is in the "Correction" note in section 5.4 and the two-fix history in section 6.4; the deeper Daily Double review is `docs/reviews/statistical-review-2026-07-23-v2.md`.)

Abstract

This paper documents the research, math, and design decisions behind a wagering-strategy tool for Jeopardy!'s two decision points that carry real stakes: the Daily Double and Final Jeopardy. It covers three things in one place: what's already known about optimal wagering (from a peer-reviewed source and from expert practitioner writing), the exact math this project built on top of that foundation, and what is proven versus what is still an educated placeholder. The intent is that a reader can verify every claim here against either a citation or a worked example, not take it on faith.

1. Motivation and scope

Jeopardy! has two moments where a contestant chooses a dollar amount rather than just answering a clue: a Daily Double (DD), found while a category is in play, and Final Jeopardy (FJ), at the end of the game. Both decisions are mathematically tractable — the legal wager bounds are fixed by the rules, the scores are known, and the outcome is binary (right or wrong) — which makes them a good target for a model, as opposed to buzzer speed or trivia knowledge, which aren't.

The scope is a 3-player game, since that's the standard format; 2-player and 1-player (practice) games are treated as reduced cases of the same underlying model, not separate logic.

Two distinct questions get asked here, and they are not the same question:

1. **"What wager can't hurt me if I'm right?"** — a guarantee against the worst case, needing no assumption about anyone's actual odds of being correct. This project calls this **Tier 1**.
2. **"Given how confident I actually am, and some sense of how opponents behave, what wager gives me the best real odds of winning?"** — a probability-weighted decision. This project calls this **Tier 2** for Final Jeopardy, and the **Daily Double engine** for the mid-game bet (which additionally has to account for however much of the board is left to play).

2. Methodology: how this was researched

The starting point was a web search for existing quantitative work on Jeopardy! wagering strategy, followed by a deliberate check on where the underlying historical game data could legally come from, since the obvious source — J-Archive, the fan-run historical database at j-archive.com — bans scraping and republication of its content in its own site terms.

That search surfaced three kinds of material, each treated differently:

- **A peer-reviewed academic source**: Tesauro, Gondek, Lenchner, Fan, and Prager (2013), published in the *Journal of Artificial Intelligence Research*. This is the IBM Watson team's own writeup of the statistical and game-theoretic strategy engine built for Watson's Jeopardy! matches. It mined roughly 3,000 historical episodes from J-Archive and published the *resulting aggregate statistics* — not the raw data — which makes those numbers citable without the reuse restriction that applies to scraped data itself. This is the paper this project leans on hardest, because it's the only source that is both rigorous and legally clean to cite from.
- **Expert practitioner writing, not peer-reviewed but influential**: Keith Williams' "The Final Wager" (a tutorial series by a former College Jeopardy! Champion) and Chuck Dupee's 1998 wagering analysis, both of which independently arrived at heuristic rules — the cover bet, the two-thirds rule — that the Tesauro paper later confirmed empirically describe how real contestants tend to bet (imperfectly).
- **Pre-scraped historical datasets on GitHub** (e.g., jwolle1's `jeopardy_clue_dataset`, 538,845 clues with Daily Double wager amounts and round-end scores). These looked, at first glance, like the ideal calibration source. Reading the dataset's own README changed that: it explicitly asks that the data not be used to build a public-facing product. That single sentence is why this project does not use that dataset (or others like it) anywhere in a shipped artifact — only, if at all, for private internal validation. See `docs/adr/0001-data-source-strategy.md` for the full reasoning.

From there, the project's design followed directly from what the Tesauro paper says about how *they* approached the same problem: rather than compute a game-theoretically perfect equilibrium (which assumes all players are perfectly rational and know it — not true of real contestants, and expensive to compute), they computed a **best-response** strategy against an empirically-fitted model of how real opponents actually wager. This project follows the same two-layer structure: a guarantee-based layer that needs no behavioral assumptions (Tier 1), and a best-response layer that does (Tier 2 / the DD engine), with the second layer's assumptions labeled as placeholders rather than presented as fact, since this project has not yet done the paper's scale of empirical fitting.

3. Data and sourcing decision

Summarized from ADR-0001 (`docs/adr/0001-data-source-strategy.md`):

Source	Used for	Restriction
J-Archive itself	Nothing (not scraped)	Site terms forbid scraping/republication
jwolle1/jeopardy_clue_dataset	Nothing shipped; internal validation only, if used at all	README forbids public-facing product use
J-Archive's own Wagering Calculator	Reference point to sanity-check against	N/A — a tool, not a data source
Tesauro et al. (2013), JAIR	Calibration numbers cited in this paper	None — published research

Decision: anything that ships — this paper, the app, any future deck — cites only the Tesauro et al. published statistics. Nothing here is built by scraping J-Archive or by embedding a restricted dataset.

4. Final Jeopardy, Tier 1: guarantee-based wagering

4.1 The question this answers

Given the scores of up to three players entering Final Jeopardy, what wager guarantees the best possible finish, assuming the worst realistic case: every opponent bets their entire legal maximum and answers correctly?

This needs no input about confidence, category, or opponent psychology — it's a minimax guarantee, not a probability estimate.

4.2 The math

Let a player's current score be `S`. The legal maximum Final Jeopardy wager is:

```
legal_max(S) = max(S, 0)
```

(A player at zero or a negative score has no wager available — there's nothing to stake.)

For any player, their **worst-case ceiling** — the highest score they could possibly reach — is what happens if they bet everything and answer correctly:

```
ceiling(S) = 2S   (if S > 0)
ceiling(S) = S    (if S <= 0, since no wager is possible)
```

For a player with several opponents, the **binding constraint** is whichever opponent has the single highest ceiling. This is the key simplification that makes the model work cleanly for three players instead of needing pairwise patchwork: because each opponent's wager and correctness are independent events, the true worst case for you is exactly "every opponent simultaneously achieves their own individual best outcome," and beating the largest of those ceilings automatically beats all the smaller ones too.

```
binding_ceiling = max(ceiling(opponent_1), ceiling(opponent_2), ...)
```

Lockout: if your current score already exceeds `binding_ceiling`, no opponent can catch you regardless of the Final Jeopardy outcome. The correct wager is $0 (or anything — it doesn't change the outcome).

Cover bet: if you're not in lockout, the minimum wager that guarantees an outright win (assuming you answer correctly) is:

```
required = binding_ceiling - your_score + 1
```

capped at your legal maximum. If `required` exceeds your legal maximum, an outright guarantee is not possible — the model reports that plainly rather than pretending otherwise.

Lock-tie: a special, favorable case. If your score is *exactly half* of the binding opponent's score, and that opponent is themselves already safe against everyone else (so they have no reason to wager anything), then betting everything and answering correctly guarantees at least a tie for first — which, under the show's actual rules, means both players are declared co-champions and both advance. This is a materially better outcome than the naive cover-bet math would suggest, and it's the kind of edge case a purely mechanical formula can miss if it isn't checked for explicitly.

4.3 Worked examples

These are the exact hand-derived cases used to test the implementation (`tests/test_fj_wager.py`), shown here so the math can be checked directly:

- **A = $12,000, B = $5,000.** `ceiling(B) = 10,000`. Since 12,000 > 10,000, A is in lockout. Correct wager: **$0**.
- **A = $9,000, B = $5,000.** `ceiling(B) = 10,000`. Not lockout. `required = 10,000 - 9,000 + 1 = 1,001`. Correct wager: **$1,001**.
- **A = $10,000, B = $6,000, C = $3,000.** `ceiling(B) = 12,000`, `ceiling(C) = 6,000`. B is the binding constraint (larger ceiling), not C, even though this is a three-player game. `required = 12,000 - 10,000 + 1 = 2,001`. Correct wager: **$2,001**. (Note this is not a pairwise patchwork of "beat B" and "beat C" separately — it's a single formula that happens to reduce to the pairwise answer here, because one opponent's ceiling dominates the other's.)
- **A = $10,000, B = $5,000, C = $1,000** (from B's perspective). B's binding opponent is A, with `ceiling(A) = 20,000` — B cannot cover that even betting everything ($5,000 max). An outright guarantee isn't possible. But: A's score (10,000) already exceeds `ceiling(C) = 2,000`, so A has no need to wager anything against C. And B's score is exactly half of A's (5,000 × 2 = 10,000). This is a lock-tie: betting everything ($5,000) and answering correctly ties A for first.

4.4 Corollary: mathematically irrelevant opponents

A direct consequence of section 4.2's "binding constraint" logic: an opponent whose ceiling never exceeds the current binding ceiling has *no effect whatsoever* on the correct wager. If a third player is far enough behind that even doubling their score plus winning couldn't reach the threshold set by whoever the actual binding opponent is, that player can be dropped from the calculation entirely with no loss of correctness — the model already behaves this way, since it only ever looks at `max(ceiling(...))` across opponents, never a sum or an average.

This means a "mathematically eliminated" rival doesn't need special-case handling in the underlying math — they're automatically ignored. What the current tool does *not* yet do is *tell the user* this explicitly at the interface level (e.g., "Rival 2's score doesn't affect this wager"), which would be a natural, low-risk usability enhancement. It's logged as a backlog item rather than built, since confirming this was a design question, not a math gap.

5. Final Jeopardy, Tier 2: confidence-weighted best response

5.1 The question this answers

Given an actual confidence level (not worst-case paranoia) and some model of how opponents tend to wager, what wager maximizes the probability of winning?

Ties for first place are treated as wins throughout this layer, matching the show's actual rule that co-champions both advance — this is a deliberate departure from Tier 1's stricter "beat them outright" framing, and it means Tier 2 can sometimes recommend a smaller, less paranoid wager than Tier 1 for an identical scoreline. Section 5.4 shows exactly this.

5.2 The math

For a candidate wager `w` and a self-confidence `p` (the probability of answering correctly), the expected win probability is computed by exact enumeration in double precision, not sampling — since the opponent wagering model used here is a small, discrete probability distribution, there's no need to introduce Monte Carlo sampling noise where the sum can be taken directly. ("Exact" here means *no sampling error*; the arithmetic is IEEE-754 double precision, whose ~1e-16 rounding is far below the 1e-9 comparison tolerance and affects no result — see risk #13.)

```
E[win | w] = p · P(win | self correct, wager w) + (1 - p) · P(win | self incorrect, wager w)
```

where each term is computed by enumerating every combination of each opponent's (wager choice × correct/incorrect outcome), weighting by the joint probability of that combination, and counting it as a win whenever the self player's resulting score ties or beats the maximum of all opponents' resulting scores in that combination.

The wager search then simply evaluates this exact expectation at a grid of candidate wagers (plus the Tier 1 cover and lock-tie wagers explicitly, in case they fall between grid points) and picks the smallest wager achieving the maximum expected win probability — smallest, because there's no reason to risk more money than necessary to reach the same expected outcome.

5.3 Where the opponent model comes from (and its honest limits)

This is the part of the model that is a placeholder, not a finished calibration, and it's documented as such in `docs/adr/0002-opponent-model-assumptions.md`. Rather than invent arbitrary percentages, the default opponent model anchors each opponent's plausible wagers on *their own* Tier 1 breakpoints (bet nothing, their own cover wager if one exists, their own lock-tie wager if one exists, bet everything) and spreads probability *uniformly* across whichever of those apply. That's a defensible "we don't know the real split, so don't pretend to" default — a maximum-entropy-style prior — not a fitted distribution. Opponent correctness is defaulted to 50%, also a placeholder.

The Tesauro paper's own data suggests the real picture is more structured than a uniform split: they found second-place bettors cluster around either "bet everything" or "bet just enough to overtake the leader," and found that third-place bettors show almost no dependence on game state at all — closer to random than strategic. Replacing the current uniform default with something closer to that structure is the natural next calibration step, and the test suite is deliberately built so that this replacement won't break anything: the hand-derived tests use explicit, fixed opponent models passed in directly, not the shipped default, so they stay valid regardless of what the default becomes.

A second, separate assumption is worth stating explicitly: the enumeration treats each opponent's *correctness* as independent of every other player's. Real Final Jeopardy correctness is positively correlated across players, because everyone faces the same clue at the same difficulty. This matters for joint events. For instance, the probability that *both* leaders miss — which is exactly how a trailing player wins when the leaders overbet — is larger under positive correlation than under the independence the model assumes. So figures this model produces for "everyone busts" style upsets (such as the 1/12 in the worked three-way example below) are best read as *conservative floors*, not over-estimates. Adding an opponent-correlation parameter is tracked as risk #12.

5.4 A worked example showing Tier 1 vs. Tier 2 diverging

Self score \$9,000, one opponent at \$5,000, opponent wagering model: bets \$0 with probability 0.5, bets everything (\$5,000) with probability 0.5, and is correct with probability 0.5 if they do wager. This gives the opponent a final-score distribution of \$5,000 (p=0.5), \$10,000 (p=0.25), \$0 (p=0.25).

- **Tier 1** says: wager at least \$1,001 to *outright beat* the opponent's \$10,000 ceiling.
- **Tier 2**, under this same opponent model and a self-confidence of 50%, finds that wagering exactly \$1,000 achieves the identical 87.5% win probability as wagering \$1,001 — because reaching exactly \$10,000 produces a *tie*, and Tier 2 counts a tie as a win, matching the real show rule.

Correction (2026-07-23). This section originally concluded that Tier 2 should prefer the \$1,000 tie over the \$1,001 strict win, on the reasoning that "betting the extra dollar adds risk for zero additional benefit." That reasoning was wrong: at 50% self-confidence there genuinely is no extra risk from the additional dollar (the *incorrect* branch is 1 dollar lower either way, which changes nothing about who it beats), and the \$1,001 wager wins outright in the *correct* branch instead of merely tying. Will caught this in the shipped app ("the smart bet results in a tie if both are correct that's dumb") when confidence was even higher (100%), where the free strict win is unambiguous. The model now uses strict-win probability (self strictly ahead of every opponent, no tie) as a secondary tiebreaker: among wagers achieving the same tie-inclusive win probability, it prefers whichever wins outright more often, then the smallest wager. For this exact example, that means **\$1,001**, not \$1,000, is now the recommended wager — see the full writeup in `docs/reviews/statistical-review-2026-07-23.md`. The underlying math describing the two wagers' outcomes above is still correct; only the *preference* between them changed.

This example is preserved (with the corrected expected wager) as `test_analyze_prefers_strict_win_over_gratuitous_tie` in `tests/test_fj_bestresponse.py`.

6. Daily Double wagering: Monte Carlo simulation

6.1 Why this needs simulation, not a closed form

Unlike Final Jeopardy, a Daily Double decision has consequences that unfold over the *rest* of the game, not just the next thirty seconds: there may be more clues to play, possibly another hidden Daily Double, and then Final Jeopardy after all of that. There's no clean closed-form "cover bet" formula for this, because the future is genuinely uncertain in a way Final Jeopardy — a single terminal event — is not. This project's approach is to simulate the rest of the game forward, many times, for each candidate wager, and see which one wins most often.

6.2 What's modeled, named explicitly

To keep the simulation honest about its own limits, every simplification is named rather than buried:

- Each remaining regular clue is treated as independently "won" by a uniformly random player with a fixed probability — the model does not simulate actual buzzer competition, and a wrong answer does not pass the clue to another player the way it would on the real show.
- Final Jeopardy, inside a simulated rollout, is resolved using the fast Tier 1 closed-form logic (section 4), not the slower Tier 2 search — running the full Tier 2 opponent-model search inside every one of thousands of simulated trials would be far too slow to be useful, and Tier 1's cover-bet logic is a reasonable stand-in for "what a rational player would roughly do" in a rollout context.
- A future, not-yet-found Daily Double (if the user indicates one is still on the board) is modeled crudely: a random player finds it and wagers a fixed fraction of their score at a fixed accuracy rate. This has not been validated and is the single roughest part of the model.

6.3 A methodological fix worth documenting: common random numbers

An early version of this simulator compared candidate wagers using independently drawn random numbers for each one. That's a subtle bug, not just an inefficiency: with a finite number of simulated trials, pure sampling noise can make a genuinely worse wager look better purely by luck, especially when two wagers are close in real quality.

The fix is a standard variance-reduction technique called **common random numbers**: for each simulated trial, the *same* underlying random sequence (who wins each remaining clue, whether a future Daily Double lands, every player's Final Jeopardy correctness) is replayed for every candidate wager being compared in that trial. Only the wager itself differs between candidates within a trial — the "luck" of the rest of the game is held fixed. This turns a noisy independent comparison into a fair, paired one, and it's what makes the engine's basic sanity checks — for example, "if you're guaranteed to answer wrong, the optimal wager is $0" — actually hold reliably instead of holding only approximately, subject to random noise.

6.4 Selecting a wager from noisy estimates: two fixes

Common random numbers (6.3) reduce sampling noise between candidates; they do not eliminate it. Every candidate wager's simulated win probability is still a Monte Carlo *estimate*. Selecting a wager from these estimates is a statistical decision, and this project got it wrong twice before getting it right — the sequence is kept here on purpose, because the second error hid underneath the first.

First error and first fix. The original engine picked "the best wager" by comparing estimates with a fixed `1e-9` tolerance, treating noisy numbers as exact. The consequence was reported directly: a two-percentage-point change in stated confidence (42% → 44%) swung the recommended wager from $350 to $4,200 for identical scores. The first fix replaced the bare argmax with a **plateau** of statistically-indistinguishable wagers, recommending the smallest one.

Second error (correction, 2026-07-23). That first fix computed the comparison standard error with the *independent* two-sample formula `sqrt(se_w^2 + se_best^2)`. That formula is wrong for this simulator, because the simulator uses common random numbers: within a trial, every wager is scored on the *same* replayed randomness, which makes the per-wager estimates strongly positively correlated. The correct variance of the difference is `Var(p_w) + Var(p_best) - 2*Cov(p_w, p_best)`, and the covariance term is exactly the variance reduction CRN exists to provide. Using the independent formula threw that reduction away, produced a spuriously wide plateau, and made the engine systematically *under-bet*: for the reported scenario at 3000 trials it recommended $100 where the CRN-correct analysis recommended $4,200. The engine's apparent stability was an artifact of a wrong-but-wide interval, not a virtue.

Second fix (current method). Two coupled changes. First, the comparison standard error is now the **paired** per-trial difference between a candidate wager and the empirical best wager — estimated directly from the per-trial win indicators, so it respects the pairing CRN induces. Second, because roughly `k` candidate wagers are each compared against the single empirical best, this is a *multiple-comparisons-with-the-best*

problem (Hsu, 1996); the band is a multiplicity-adjusted Bonferroni value `z* = Phi^{-1}(1 - alpha/k)` (default `alpha = 0.05`) rather than a per-comparison `z`. The recommendation is the smallest wager on the resulting plateau.

This resolves both failure modes at once. On a flat scenario, where all wagers are genuinely about equal, it recommends $0, stably across seeds and across neighbouring confidences — the right call because the surface really is flat. On a clear-signal scenario (guaranteed correct, where betting genuinely helps), it recommends a *positive* wager ($1,200 in the worked case), stably across seeds — the paired standard error detects that smaller wagers are significantly worse, so the engine does not collapse to $0 everywhere. Full method, evidence (including the measured independent-vs-paired standard errors and correlations), and validation are in `docs/reviews/statistical-review-2026-07-23-v2.md`; the two errors are tracked as risks #9 and #10 in `RISK_REGISTER.md`.

One honesty note carried from that review: the reported win-probability percentage is the in-sample maximum over candidate wagers, so it is a mild *optimistic* (selection-biased) estimate. The recommended *wager* is well insulated from this, because it is the smallest wager on a multiplicity-controlled plateau, not the raw argmax — but the percentage should be read as an upper bound, not an unbiased point estimate (risk #11).

6.5 Legal wager bounds

A Daily Double wager may be up to the greater of the player's current score or the maximum clue value showing on the board for that round: $1,000 in Single Jeopardy, $2,000 in Double Jeopardy, under the rules current as of this writing.

7. Validation

Every module has automated tests (33 across the four engine modules as of this writing). Two different testing strategies are used depending on whether hand-derivation is possible:

- **Tier 1 and the exact-expectation parts of Tier 2** can be checked against numbers worked out by hand beforehand (shown in the test comments), not captured from the code's own output — this is the stronger standard, since it can catch the implementation being wrong in a way that's self-consistent but incorrect.
- **The Monte Carlo Daily Double engine** cannot be hand-derived exactly (it's inherently stochastic), so its tests check invariants instead: legal wager bounds, that a guaranteed-wrong bet always recommends $0, that a guaranteed-correct bet is never penalized for betting more, determinism given a fixed random seed, that every probability stays within [0, 1], that the paired comparison standard error never exceeds the independent two-sample formula (the invariant that proves the CRN pairing is actually used — see section 6.4), that the Bonferroni band grows with the number of candidate wagers, and that a clear-signal scenario still recommends a positive, seed-stable wager rather than collapsing to $0.

A separate, standalone JavaScript port of the same logic powers the interactive browser app (`app/index.html`). It is re-checked by extracting its core logic and running it under Node against the Python test suite's hand-derived values (the exact Tier 1 and Tier 2 numbers), plus the inverse-normal and Bonferroni-band reference values and the paired-standard-error invariant — every case matches. Note one hard limit on this parity: the Python engine and the JS app use *different* random number generators (Mersenne Twister vs. mulberry32), so their Monte Carlo Daily Double outputs are only distributionally comparable, never identical trial-for-trial; only the exact and deterministic functions can be checked for equality. The port is also not part of the automated test suite, so it can drift if either side changes without a manual re-check. Tracked as risk #8 in `RISK_REGISTER.md`.

8. Known limitations

In order of how much they matter:

1. **The Tier 2 and Daily Double opponent-behavior numbers are placeholders**, not fitted to real historical data (section 5.3). Their *shape* of recommendation (don't overbet past the point of no added

benefit, prefer smaller wagers among ties) is sound; their *exact* win-probability outputs shouldn't be quoted as validated statistics.

2. **The Daily Double simulator's model of the rest of the game is simplified** (section 6.2) — it ignores buzzer competition dynamics entirely. Results should be treated as directionally useful, especially for the very next decision, and treated more skeptically the further out a rollout has to project.
3. **Opponent correctness is modeled as independent across players** (section 5.3), but in reality it is positively correlated. This makes the model's "everyone busts" upset probabilities conservative floors rather than over-estimates (risk #12).
4. **The reported Daily Double win probability is selection-biased upward** (section 6.4) — it is the in-sample maximum over candidate wagers. The recommended *wager* is well insulated; the reported *percentage* is a mild optimistic upper bound (risk #11).
5. **The JavaScript app is a manually maintained port** with a *different* PRNG from the Python engine, so its Monte Carlo Daily Double numbers are only distributionally comparable, and it could drift without an automated check (section 7, risk #8).
6. **This tool is scoped as a study and post-game analysis aid, not a live-assist device.** Using it during an actual taping would violate the show's rules; that's a hard boundary, not a data limitation.

9. References

- Tesauro, G., Gondek, D., Lenchner, J., Fan, J., & Prager, J. M. (2013). Analysis of Watson's strategies for playing Jeopardy!. *Journal of Artificial Intelligence Research*, 47, 205–251. https://arxiv.org/pdf/1402.0571
- Williams, K. "How to wager in Final Jeopardy!" *The Final Wager*. https://thefinalwager.wordpress.com/tutorial/
- Dupee, C. (1998). Wagering strategy analysis for Final Jeopardy!, as cited and summarized in Tesauro et al. (2013).
- Hsu, J. C. (1996). *Multiple Comparisons: Theory and Methods*. Chapman & Hall. (Multiple-comparisons-with-the-best, applied to the Daily Double wager selection — section 6.4.)
- jwolle1. jeopardy_clue_dataset [dataset]. GitHub. Referenced for its reuse restriction, not used as a data source in this project. https://github.com/jwolle1/jeopardy_clue_dataset
- J! Archive Wagering Calculator. https://j-archive.com/wageringcalculator.php

Related project documents

- `docs/adr/0001-data-source-strategy.md` — full data-sourcing decision record
- `docs/adr/0002-opponent-model-assumptions.md` — full opponent-model decision record
- `docs/explanation/wagering-theory-research.md` — the original research memo this paper consolidates
- `docs/explanation/tiers-explained.md` — a plain-language companion to this paper's Tier 1/Tier 2 discussion
- `docs/reviews/statistical-review-2026-07-23-v2.md` — the referee-style second review that produced section 6.4's second fix and the limitations catalog
- `docs/reviews/statistical-review-2026-07-23.md` — full method and findings behind the section 5.4 correction and section 6.4 fix
- `docs/tutorial/`, `docs/how-to/`, `docs/reference/` — task-oriented documentation (Diátaxis), for using and maintaining the tool rather than understanding its theory
- `RISK_REGISTER.md` — the live, tracked list of open risks referenced throughout this paper
- `CHANGELOG.md` — version history of every engine and app change